

Vertex AI：在同一個 VM 託管模型，以便預測結果

來源：<https://codelabs.developers.google.com/vertex-cohost-prediction?hl=zh-tw>

步驟數：7

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [用途總覽](#)
4. [設定環境](#)
5. [訓練模型](#)
6. [部署模型](#)
7. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將使用 [Vertex AI](#) 的[共同託管模型功能](#)，在同一個 VM 託管多個模型，以便進行線上預測。

課程內容

內容如下：

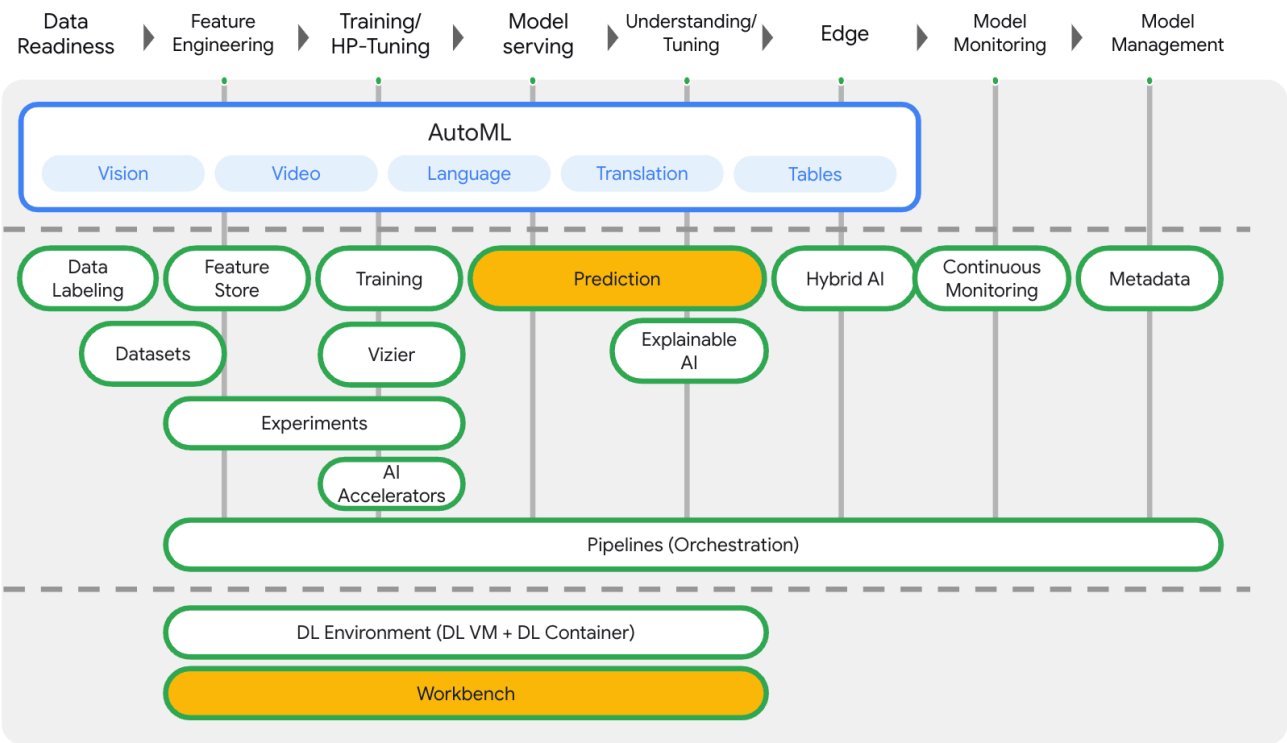
- 建立 `DeploymentResourcePool`
- 在 `DeploymentResourcePool` 中部署模型

在 Google Cloud 上執行這個實驗室的總費用約為 **\$2 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。如有任何意見，請參閱[支援頁面](#)。

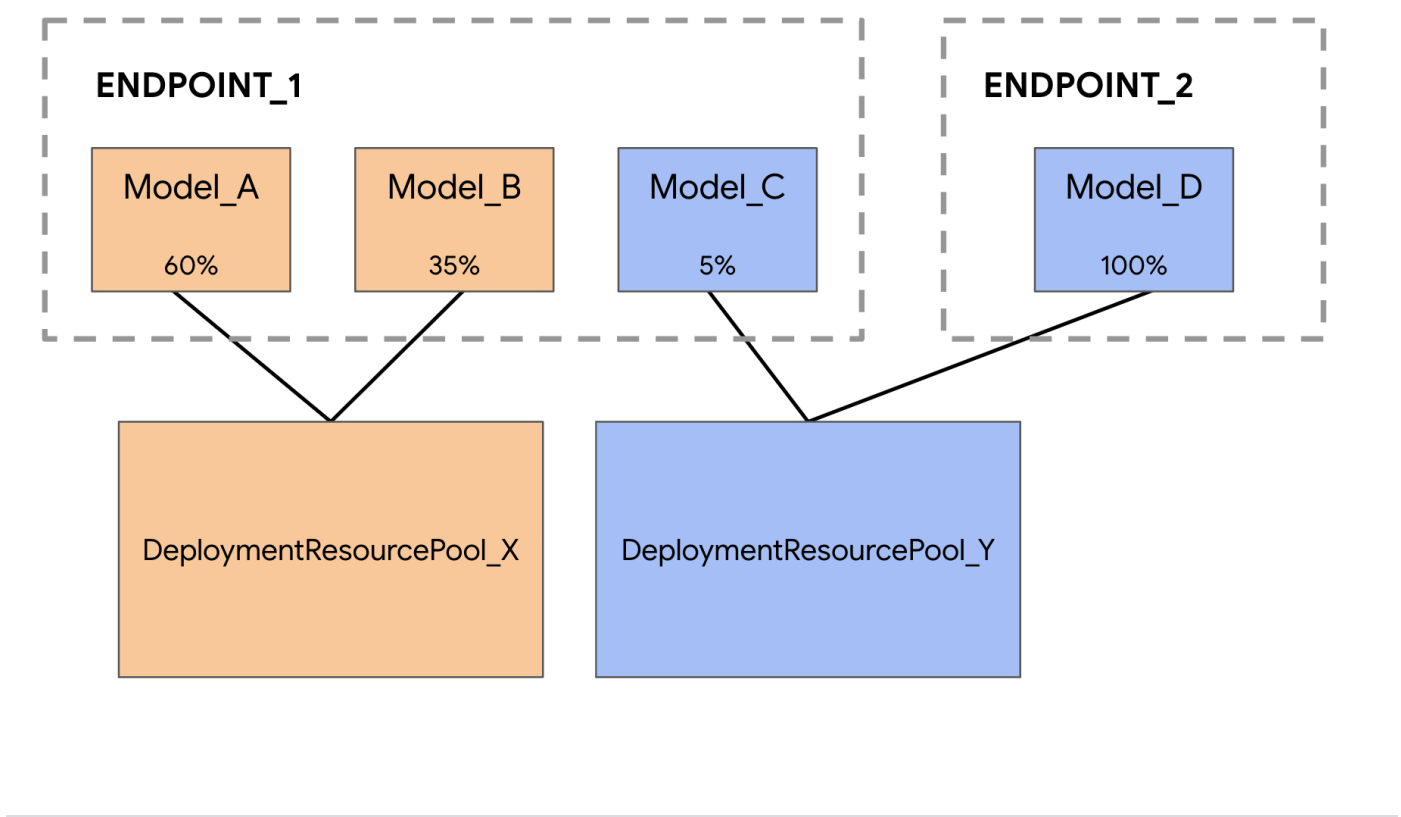
Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於下列產品：**Predictions**和**Workbench**



3. 用途總覽

將模型部署至 [Vertex AI 預測服務](#)時，每個模型預設都會部署至專屬 VM。如要提高主機的成本效益，您可以在同一部 VM 上代管多個模型，進而提升記憶體和運算資源的使用率。您選擇部署至同一部 VM 的模型數量取決於模型大小和流量模式，但如果您部署了許多模型，且流量稀疏，這項功能就特別實用。

共同託管模型支援功能導入了「部署資源集區」的概念，可將模型分組，在 VM 中共用資源。如果模型共用端點，或部署至不同端點，都可以共用 VM。目前，同一資源集區中的模型必須使用相同的容器映像檔，包括 Vertex Prediction 預建容器的架構版本。此外，這個版本僅支援使用 TensorFlow 模型架構的 [Vertex Prediction 預先建構容器](#)，其他模型架構和自訂容器目前不支援。



步驟 4 · 約 10 分鐘

4. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。

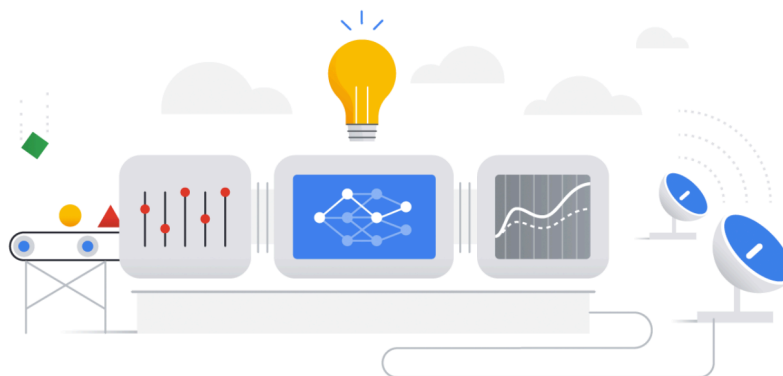
步驟 2：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

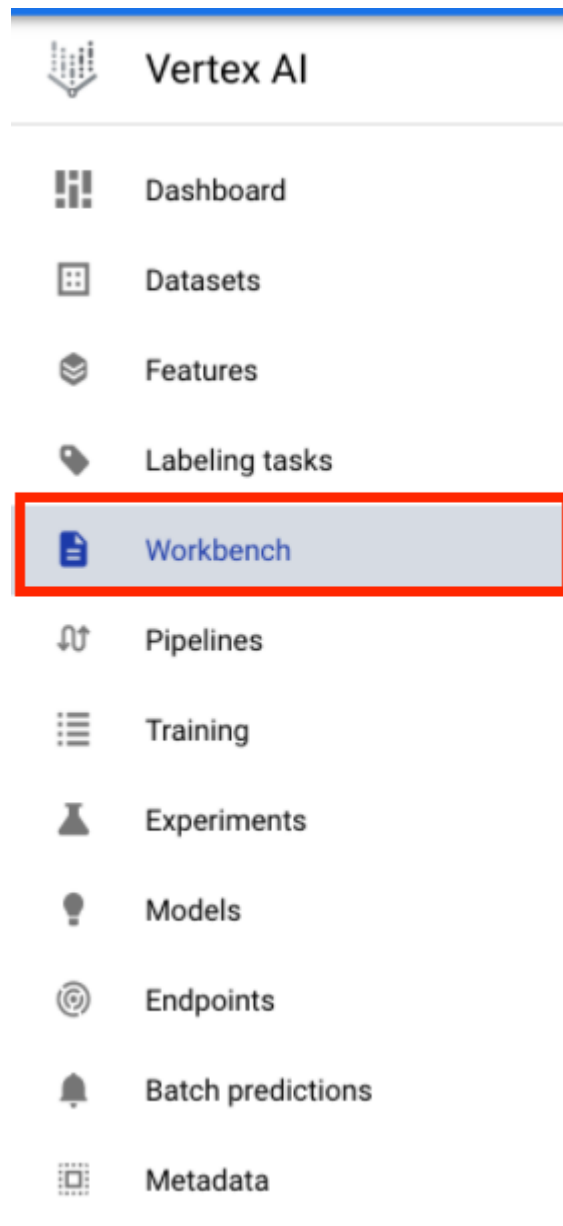
Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

+ CREATE BATCH PREDICTION

步驟 3：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



如果尚未啟用 Notebooks API，請先啟用。



Notebooks API

Google Enterprise API

Notebooks API is used to manage notebook resources in Google Cloud.

ENABLE

TRY THIS API [↗](#)

OVERVIEW

PRICING

DOCUMENTATION

Overview

Notebooks API is used to manage notebook resources in Google Cloud.

Additional details

Type: [SaaS & APIs](#)

Last updated: 15/09/2021

Category: [Google Enterprise APIs](#)

Service name: notebooks.googleapis.com

啟用後，按一下「MANAGED NOTEBOOKS」(代管型筆記本)：

Managed notebooks

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[■ STOP](#)

[⏻ RESET](#)

[🗑 DELETE](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

EXECUTIONS [PREVIEW](#)

SCHEDULES [PREVIEW](#)

Managed notebooks provide JupyterLab services and flexible computing resources integrated with Google Cloud services. [Learn more](#)

然後選取「新增記事本」。

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

Notebooks service has been moved under the Vertex AI Workbench

為筆記本命名，然後在「權限」下方選取「服務帳戶」

← Create a managed notebook

Notebook name *

my-notebook

63 character limit with lowercase letters, digits or '-' only. Must start with a letter. Cannot end with a '-'.

Region

us-central1 (Iowa)



Permission

JupyterLab access modes determine who can use a notebook instance and which credentials are used to call Google APIs. **This cannot be changed once the notebook is created.**

☒ Service account

Service account mode allows anyone who is granted the `iam.serviceAccounts.actAs` permission on the specified service account to access the JupyterLab UI. [Learn more](#)

☐ Single user only

Single-user-only mode restricts access to the user specified below. [Learn more](#)

☒ Use Compute Engine default service account

選取「進階設定」。

在「安全性」下方，選取「啟用終端機」(如果尚未啟用)。

Security

☒ Enable nbconvert

☒ Enable file downloading from Notebook UI

☒ Enable terminal

其他進階設定可以保留原樣。

接著點選「建立」。執行個體會在幾分鐘內佈建完畢。

建立執行個體後，請選取「OPEN JUPYTERLAB」。

☐	☒	my-notebook	OPEN JUPYTERLAB
--------------------------	-------------------------------------	-------------	-----------------

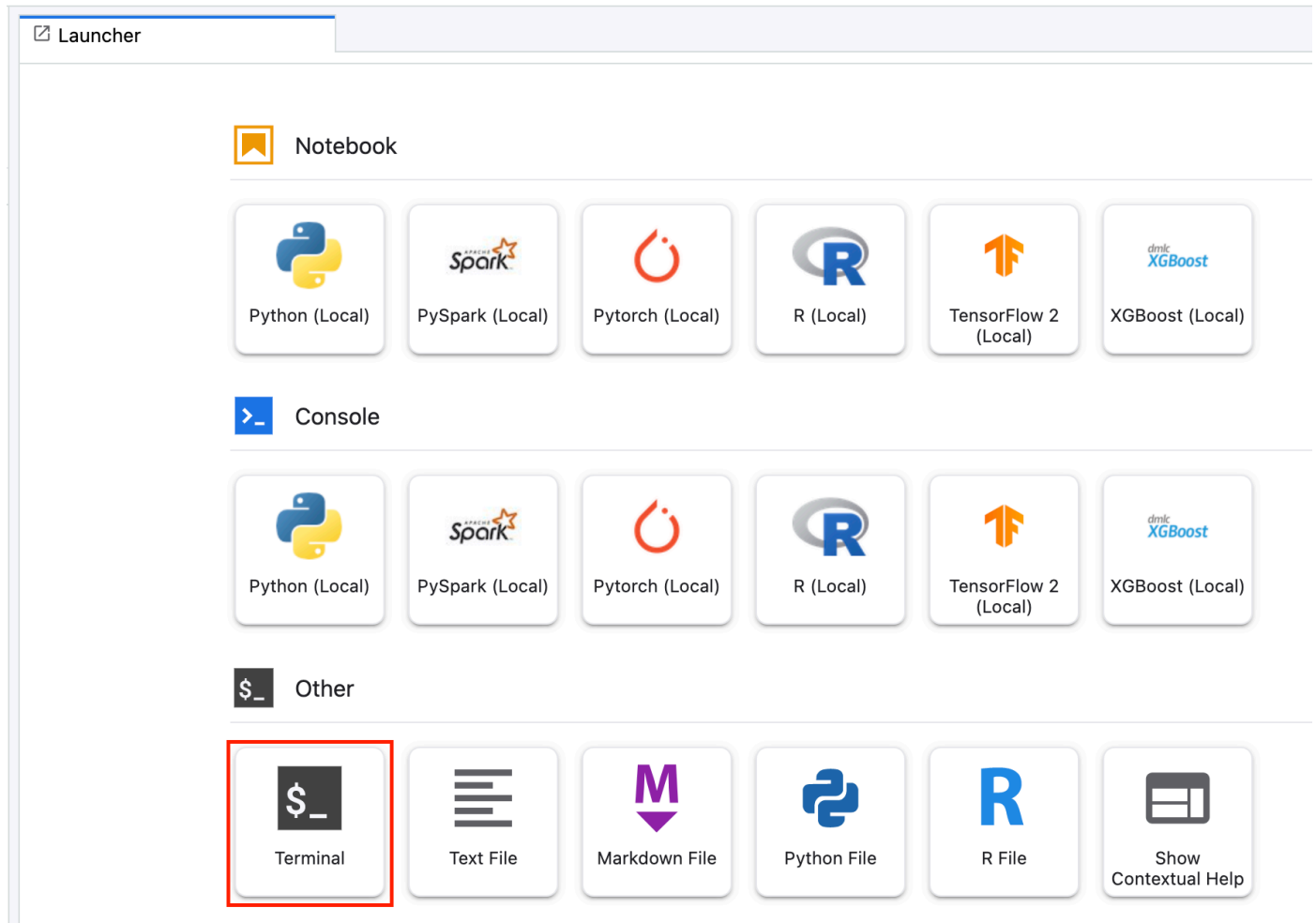
步驟 5 · 約 10 分鐘

5. 訓練模型

試用共同主辦功能前，我們需要先訓練模型，並將儲存的模型構件存放在 Cloud Storage bucket 中。我們將使用 [Workbench 筆記本執行器](#) 啟動訓練工作。

步驟 1：建立 Cloud Storage 值區

如果專案中已有要使用的現有 bucket，可以略過這個步驟。否則，請從啟動器開啟新的終端機工作階段。



在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

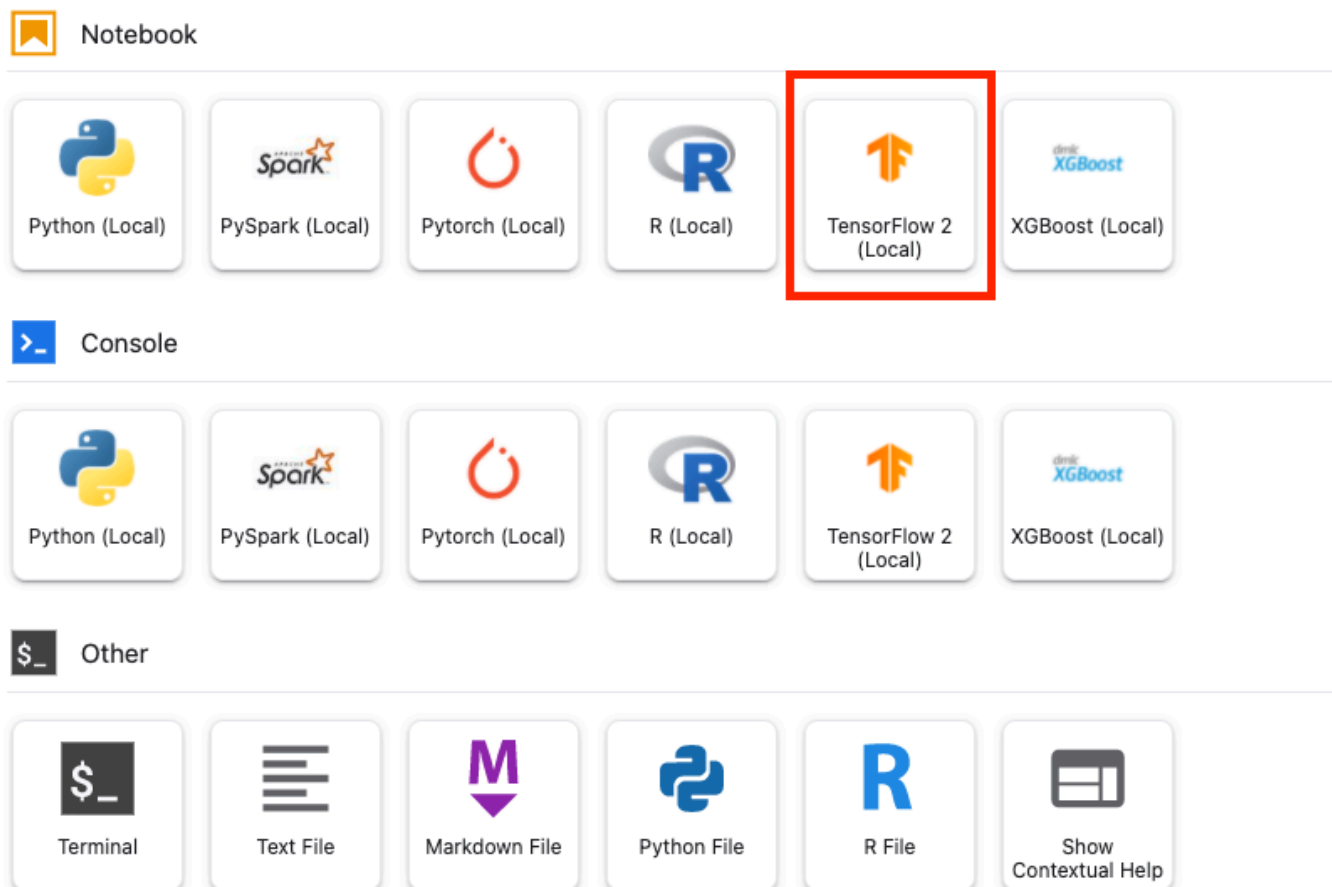
```
PROJECT_ID='your-cloud-project'
```

接著，執行下列指令，在專案中建立新的 bucket。

```
BUCKET="gs://${PROJECT_ID}-bucket"  
gsutil mb -l us-central1 $BUCKET
```

步驟 2：啟動筆記本執行作業

從 Workbench 執行個體的啟動器開啟新的 TensorFlow 2 筆記本。



以下程式碼會在 [IMDB 電影評論資料集](#) 上訓練二元情緒分類器 (正面或負面)。將程式碼貼到筆記本中。

請務必將 `{YOUR_BUCKET}` 替換為上一步建立的 bucket (或專案中的另一個 bucket)。我們會在這裡儲存已儲存的模型構件，稍後將模型上傳至 [Vertex AI Model Registry](#) 時會用到。

```

import numpy as np

import tensorflow_datasets as tfds
import tensorflow as tf

# REPLACE WITH YOUR BUCKET!
OUTPUT_PATH='gs://{YOUR_BUCKET}/model_output'

BUFFER_SIZE = 10000
BATCH_SIZE = 64
VOCAB_SIZE = 1000

# Load data
dataset, info = tfds.load('imdb_reviews', with_info=True,
                           as_supervised=True)
train_dataset, test_dataset = dataset['train'], dataset['test']

train_dataset =
train_dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
test_dataset = test_dataset.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

# Create text encoder
encoder = tf.keras.layers.TextVectorization(
    max_tokens=VOCAB_SIZE)
encoder.adapt(train_dataset.map(lambda text, label: text))

# Create model
model = tf.keras.Sequential([
    encoder,
    tf.keras.layers.Embedding(
        input_dim=len(encoder.get_vocabulary()),
        output_dim=64,
        # Use masking to handle the variable sequence lengths
        mask_zero=True),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(1)
])

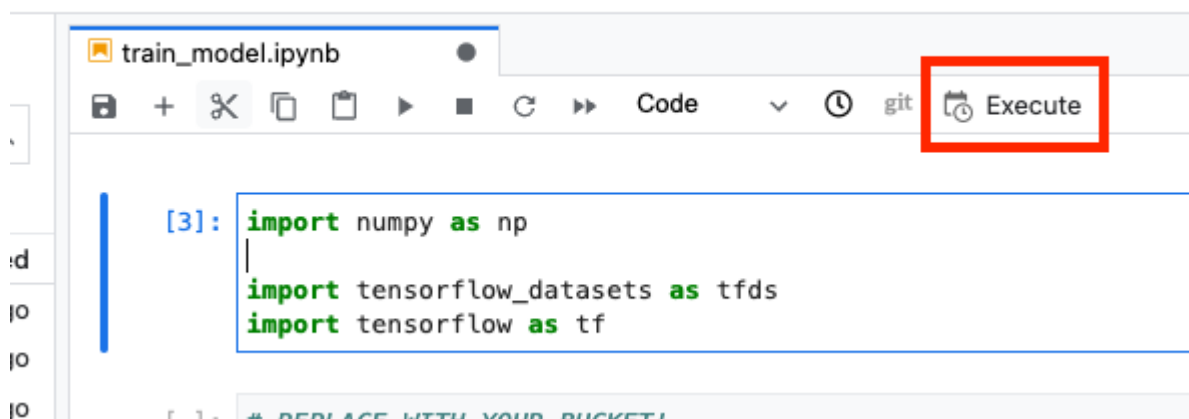
# Compile model
model.compile(loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
              optimizer=tf.keras.optimizers.Adam(1e-4),
              metrics=['accuracy'])

# Fit model
history = model.fit(train_dataset, epochs=10,
                    validation_data=test_dataset,
                    validation_steps=30)

# Save model
model.save(OUTPUT_PATH)

```

接著選取「執行」按鈕。



然後依下列方式設定執行作業，並按一下「提交」

Submit notebooks to Executor

Create a one-time or recurring notebook execution using Vertex AI Training. Results are stored in a sharable Cloud Storage bucket. Charges apply for training and storage. [Learn more](#)

Notebook: train_model.ipynb

Execution name
train_model

Up to 128 lowercase letters, numbers, or underscores.

Machine type
n1-standard-4 (4 CPUs, 15 GB RAM)

Accelerator type
NVIDIA Tesla T4

Accelerator count
1

Environment
TensorFlow Enterprise 2.7 (GPU)

Type
One-time execution

Execution will start immediately after being submitted

✓ ADVANCED OPTIONS

您可以在控制台的「執行」分頁中，追蹤訓練工作的狀態。

Workbench

REFRESH

DELETE

MANAGED NOTEBOOKS

USER-MANAGED NOTEBOOKS

EXECUTIONS

SCHEDULES

This page shows your notebook executions created through Notebook Executor. You can share execution results with others by clicking the  share icon in the table below.

[Learn more](#)

Region

us-central1 (Iowa)

☐	Execution name	Source Notebook
☐	train_model VIEW RESULT	train_model_at_2022_06_02_10_21_53.ipynb

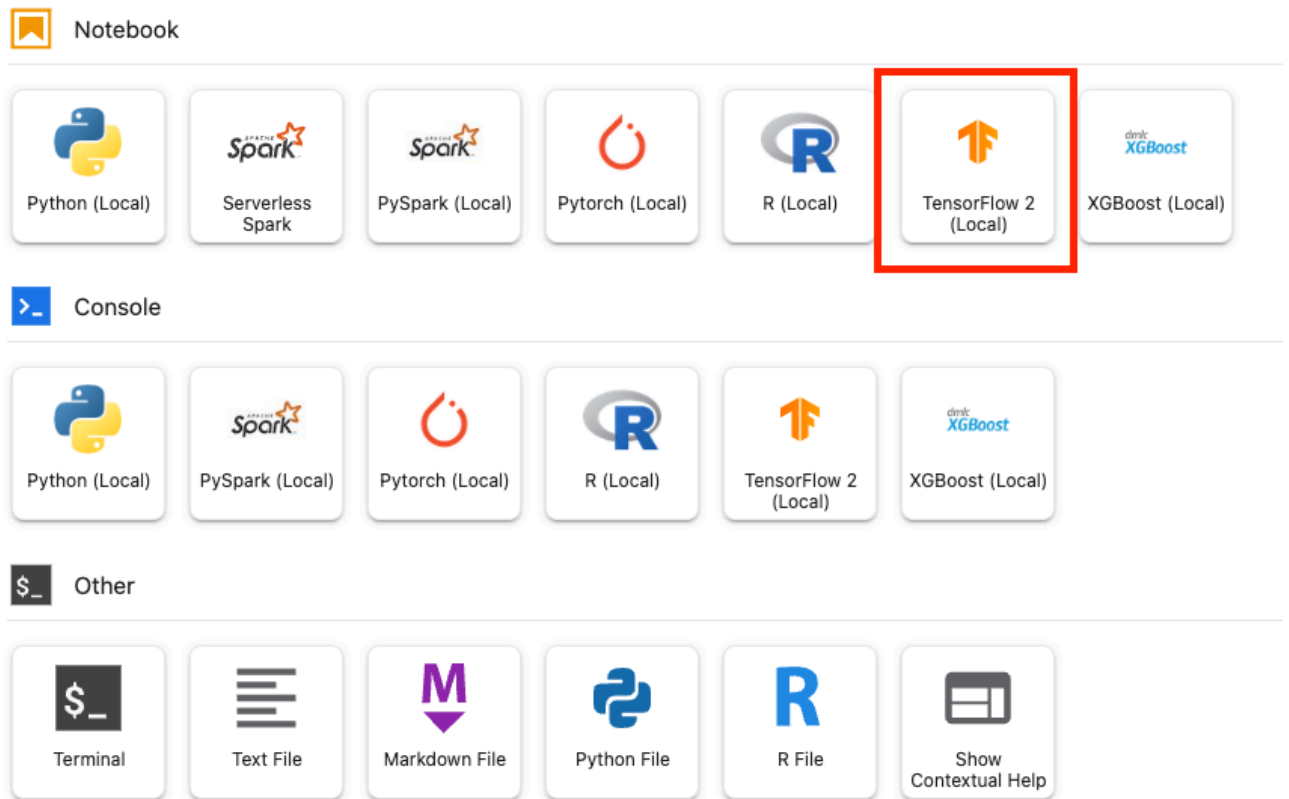
模型會在 7 分鐘內訓練完成

步驟 6 · 約 20 分鐘

6. 部署模型

步驟 1：上傳模型

執行完成後，請返回 Workbench 筆記本上傳模型。建立新的 TensorFlow 筆記本。



首先，匯入 Vertex AI Python SDK

```
from google.cloud import aiplatform
```

然後上傳模型，並將 `{YOUR_BUCKET}` 替換為您在訓練程式碼中指定的 bucket。

```
# replace {YOUR_BUCKET}
model_1 = aiplatform.Model.upload(display_name='text-model-1',
                                   artifact_uri='gs://{YOUR_BUCKET}/model_output',
                                   serving_container_image_uri='us-docker.pkg.dev/vertex-ai/prediction/tf2-cpu.2-8:latest')
```

為示範起見，我們上傳這個模型兩次，在 Vertex AI 中建立兩個不同的模型資源。這樣我們才能測試將多個模型部署至部署資源集區內的單一端點。在實際情況中，您會使用兩個不同的模型，而不是從相同的已儲存構件建立模型，但這是捷徑，因此我們不必啟動其他訓練執行作業。此外，您也可以選擇將這兩個模型部署至同一個部署資源集區中的不同端點。


```
# replace {YOUR_BUCKET}
model_2 = aiplatform.Model.upload(display_name='text-model-2',
                                  artifact_uri='gs://{YOUR_BUCKET}/model_output',
                                  serving_container_image_uri='us-docker.pkg.dev/vertex-ai/prediction/tf2-cpu.2-8:latest')
```

現在您應該會在 Vertex AI Model Registry 中看到這兩個模型。由於我們尚未部署模型，因此部署狀態為空白。

Models + CREATE 📁 IMPORT				
Models are built from your datasets or unmanaged data sources. There are many different types of machine learning models available on Vertex AI, depending on your use case and level of experience with machine learning. Learn more				
Region us-central1 (Iowa) ▼ ?				
≡ Filter Enter a property name				
Name	Deployment status	Description	Default version	Type
text-model-2	—	—	1	⌕ Imported
text-model-1	—	—	1	⌕ Imported

步驟 2：建立端點

建立端點。請注意，這與將模型部署至端點不同。

```
endpoint = aiplatform.Endpoint.create('cohost-endpoint')
```

端點建立完成後，您會在控制台中看到該端點。

Endpoints + CREATE ENDPOINT				
Endpoints are machine learning models made available for online prediction requests. Endpoints are useful for timely predictions from many users (for example, in response to an application request). You can also request batch predictions if you don't need immediate results.				
To create an endpoint, you need at least one machine learning model. Learn more				
Region us-central1 (Iowa) ▼ ?				
≡ Filter Enter a property name				
☐ Name	ID	Status	Models	
☐ cohost-endpoint	5646555158583181312	✓ Ready	0	

步驟 3：建立 DeploymentResourcePool

您可以使用下列指令建立 DeploymentResourcePool。請務必將 {YOUR_PROJECT} 替換成專案 ID。

```
# replace {YOUR_PROJECT}
PROJECT_ID={YOUR_PROJECT}
REGION="us-central1"
VERTEX_API_URL=REGION + "-aiplatform.googleapis.com"
VERTEX_PREDICTION_API_URL=REGION + "-prediction-aiplatform.googleapis.com"
MULTI_MODEL_API_VERSION="v1beta1"

# Give the pool a name
DEPLOYMENT_RESOURCE_POOL_ID="my-resource-pool"

import json
import pprint
pp = pprint.PrettyPrinter(indent=4)

CREATE_RP_PAYLOAD = {
    "deployment_resource_pool":{
        "dedicated_resources":{
            "machine_spec":{
                "machine_type":"n1-standard-4"
            },
            "min_replica_count":1,
            "max_replica_count":2
        }
    },
    "deployment_resource_pool_id":DEPLOYMENT_RESOURCE_POOL_ID
}
CREATE_RP_REQUEST=json.dumps(CREATE_RP_PAYLOAD)
pp.pprint("CREATE_RP_REQUEST: " + CREATE_RP_REQUEST)

!curl \
-X POST \
-H "Authorization: Bearer $(gcloud auth print-access-token)" \
-H "Content-Type: application/json" \
https://{VERTEX_API_URL}/{MULTI_MODEL_API_VERSION}/projects/{PROJECT_ID}/locations/{REGION}/deploymentResourcePools \
-d '{CREATE_RP_REQUEST}'
```

您可以執行下列指令查看集區：

```
!curl -X GET \
-H "Authorization: Bearer $(gcloud auth print-access-token)" \
-H "Content-Type: application/json" \
https://{VERTEX_API_URL}/{MULTI_MODEL_API_VERSION}/projects/{PROJECT_ID}/locations/{REGION}/deploymentResourcePools/{DEPLOYMENT_RESOURCE_POOL_ID}
```

步驟 4：將模型部署至端點

資源集區建立完成後，我們就能在資源集區中部署模型。

首先，我們會部署 `model_1`。請務必將 `MODEL_1_ID` 和 `ENDPOINT_ID` 替換為對應的 ID。

您可以執行 `model_1.name` 取得模型 ID，並執行 `endpoint.name` 取得端點 ID。

```
MODEL_1_ID="{MODEL_1_ID}"
ENDPOINT_ID="{ENDPOINT_ID}"
```

下列指令會將 `model_1` 部署至資源集區內的端點。

```
MODEL_NAME =
"projects/{project_id}/locations/{region}/models/{model_id}".format(project_id=PROJECT_ID,
region=REGION, model_id=MODEL_1_ID)
SHARED_RESOURCE =
"projects/{project_id}/locations/{region}/deploymentResourcePools/{deployment_resource_pool_id}".format(project_id=PROJECT_ID, region=REGION,
deployment_resource_pool_id=DEPLOYMENT_RESOURCE_POOL_ID)

DEPLOY_MODEL_PAYLOAD = {
    "deployedModel": {
        "model": MODEL_NAME,
        "shared_resources": SHARED_RESOURCE
    },
    "trafficSplit": {
        "0": 100
    }
}
DEPLOY_MODEL_REQUEST=json.dumps(DEPLOY_MODEL_PAYLOAD)
pp.pprint("DEPLOY_MODEL_REQUEST: " + DEPLOY_MODEL_REQUEST)

!curl -X POST \
-H "Authorization: Bearer $(gcloud auth print-access-token)" \
-H "Content-Type: application/json" \
https://{VERTEX_API_URL}/{MULTI_MODEL_API_VERSION}/projects/{PROJECT_ID}/locations/{REGION}/endpoints/{ENDPOINT_ID}:deployModel \
-d '{DEPLOY_MODEL_REQUEST}'
```

這項作業需要幾分鐘才能完成，完成後，您會在控制台中看到模型已部署至端點。

[← cohost-endpoint](#) [EDIT SETTINGS](#) [SAMPLE REQUEST](#)

Region
us-central1

Logs
[View Logs](#)

☒ Model	Status	Most recent alerts	Monitoring	Traffic split	Compute node
☒ text-model-1 (Version 1)	✓ Ready	—	Disabled	100%	—

DEPLOY ANOTHER MODEL

Chart interval: ☒ 1 hour 6 hours 12 hours 1 day 2 days 4 days 7 days 14 days 30 days

PERFORMANCE RESOURCE USAGE

Predictions/second

接著，我們可以在同一個部署集區中部署 `model_2`。我們會將其部署至與 `model_1` 相同的端點。不過，您也可以選擇將 `model_2` 部署至同一個資源集區中的其他端點。

將 `MODEL_ID` 更新為 `model_2` 的 ID。同樣地，您可以執行 `model_2.name` 取得這個 ID

```
MODEL_2_ID="{MODEL_2_ID}"
```

然後部署 `model_2`。由於我們已將 `model_1` 部署至端點，因此需要更新 `trafficSplit`，以便在兩個模型之間分配流量。如果選擇將 `model_2` 部署到同一資源集區中的其他端點，我們不必更新 `trafficSplit`。

如要更新流量分配，您需要為 `model_1` 定義 `DeployedModel ID`。請注意，這與模型 ID 不同。

您可以執行 `endpoint.list_models()[0].id` 取得已部署的模型 ID

```
DEPLOYED_MODEL_1_ID = {DEPLOYED_MODEL_1_ID}
```

然後執行下列指令，部署第二個模型。

```

MODEL_NAME =
"projects/{project_id}/locations/{region}/models/{model_id}".format(project_id=PROJECT_ID,
region=REGION, model_id=MODEL_2_ID)
SHARED_RESOURCE =
"projects/{project_id}/locations/{region}/deploymentResourcePools/{deployment_resource_pool_id}".format(project_id=PROJECT_ID, region=REGION,
deployment_resource_pool_id=DEPLOYMENT_RESOURCE_POOL_ID)

#`trafficSplit` is a map from a DeployedModel's ID to the percentage of this Endpoint's
traffic that should be forwarded to that DeployedModel.
# The traffic percentage values for an endpoint must add up to 100.
# The key for the model being deployed is "0".

DEPLOY_MODEL_PAYLOAD = {
    "deployedModel": {
        "model": MODEL_NAME,
        "shared_resources": SHARED_RESOURCE
    },
    "trafficSplit": {
        "0": 50,
        DEPLOYED_MODEL_1_ID: 50
    }
}
DEPLOY_MODEL_REQUEST=json.dumps(DEPLOY_MODEL_PAYLOAD)
pp.pprint("DEPLOY_MODEL_REQUEST: " + DEPLOY_MODEL_REQUEST)

!curl -X POST \
-H "Authorization: Bearer $(gcloud auth print-access-token)" \
-H "Content-Type: application/json" \
https://{VERTEX_API_URL}/{MULTI_MODEL_API_VERSION}/projects/{PROJECT_ID}/locations/{REGION}/endpoints/{ENDPOINT_ID}:deployModel \
-d '{DEPLOY_MODEL_REQUEST}'

```

同樣地，在這個範例中，兩個模型都部署到同一個端點，但您也可以將模型共同代管到同一個資源集區，並部署到不同的端點。這樣就不必擔心流量拆分問題。

部署第二個模型後，您會在控制台中看到這兩個模型。

← cohost-endpoint EDIT SETTINGS SAMPLE REQUEST						
Region us-central1		Logs View Logs				
☒	Model	Status	Most recent alerts	Monitoring	Traffic split	Cor
☒	text-model-1 (Version 1)	🟢 Ready	—	Disabled	50%	—
☒	text-model-2 (Version 1)	🟢 Ready	—	Disabled	50%	—
DEPLOY ANOTHER MODEL						

步驟 5：取得預測結果

最後一步是測試端點並取得預測結果。

首先，請定義測試句子。

```
x_test=['The movie was cool. The animation and the graphics were out of this world. I would  
recommend this movie.']
```

接著，在端點上呼叫預測，系統會從部署至端點的模型傳回預測結果。

```
endpoint.predict(instances=x_test)
```

🎉 恭喜！🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

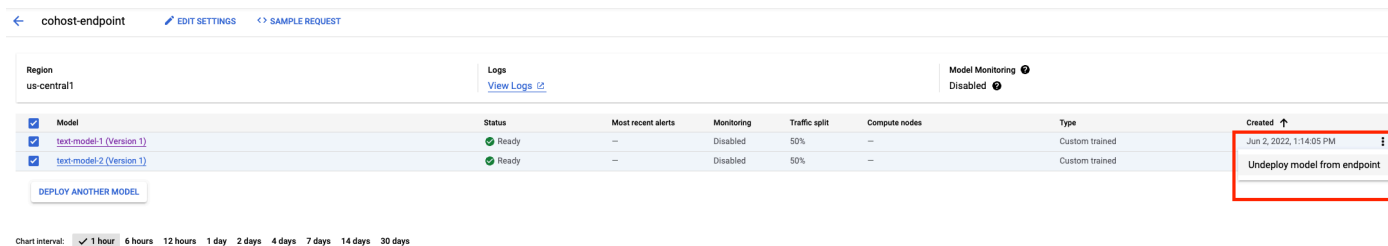
- 在同一個 VM 託管模型，以便進行線上預測

如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

步驟 7 · 約 1 分鐘

7. 清除所用資源

如果沒有使用模型的打算，建議從端點取消部署模型。您也可以完全刪除端點。如有需要，您隨時可以將模型重新部署至端點。



cohost-endpoint

Region: us-central1

Logs: View Logs

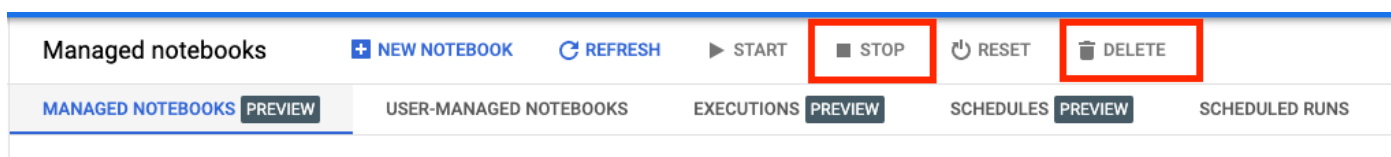
Model Monitoring: Disabled

Model	Status	Most recent alerts	Monitoring	Traffic split	Compute nodes	Type	Created
text-model-1 (Version 1)	Ready	—	Disabled	50%	—	Custom trained	Jun 2, 2022, 1:14:05 PM
text-model-2 (Version 1)	Ready	—	Disabled	50%	—	Custom trained	

DEPLOY ANOTHER MODEL

Chart interval: 1 hour, 6 hours, 12 hours, 1 day, 2 days, 4 days, 7 days, 14 days, 30 days

Workbench 管理的筆記本會在閒置 180 分鐘後自動逾時，因此您不必擔心執行個體會關閉。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。

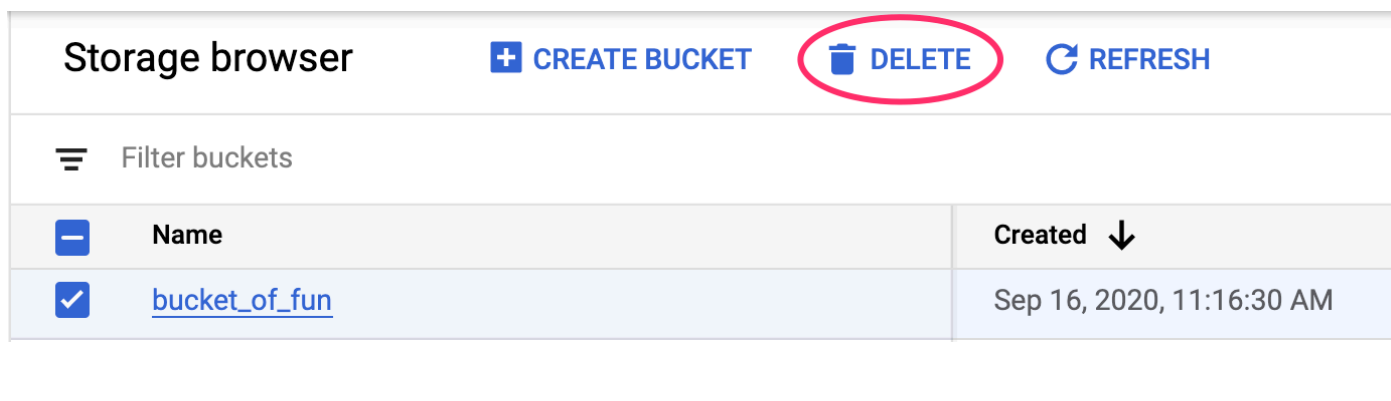


Managed notebooks

NEW NOTEBOOK REFRESH START STOP RESET DELETE

MANAGED NOTEBOOKS PREVIEW USER-MANAGED NOTEBOOKS EXECUTIONS PREVIEW SCHEDULES PREVIEW SCHEDULED RUNS

如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：



Storage browser

CREATE BUCKET DELETE REFRESH

Filter buckets

Name	Created
bucket_of_fun	Sep 16, 2020, 11:16:30 AM